Computer Networks Lab (CSE/PC/B/S/314): Assignment 3 Report

Medium Access Control: CSMA Techniques on an Emulated Shared Channel

1. Title

Field	Details
Name	Soumyadip Modak (302510501001)
Class	(BCSE- III /3rd Year / 1stSemester)
Group	A1
Assignment number	3 (CO3)
Course	CSE/PC/B/S/314 Computer Networks Lab
Code demonstration deadline	21-25 September 2026
Report submission deadline	
Date of submission	
Language / platform	C, POSIX UDP sockets (Linux)

Problem statement. Emulate a shared channel that keeps its state as IDLE, BUSY or COLLISION, and run several independent sender processes as network stations. Each station senses the channel and tries to transmit according to a CSMA strategy: Non-Persistent, 1-Persistent, p-Persistent or CSMA/CD. Write the functions for carrier sensing, collision handling and back-off, and analyse how the number of contending stations affects the number of collisions, the average transmission delay and the channel throughput. For p-Persistent CSMA, plot the three metrics against p for a fixed number of stations. For all schemes, plot them against the number of stations N and compare the schemes.

2. Design

2.1 Purpose of the program

The program has three kinds of process connected over UDP. The **Channel (Channel.c)** emulates the shared medium and keeps its state. Several **Sender (Sender.c)** processes act as stations: each reads the test file, builds frames, senses the channel with sense_channel(), and transmits through csma_sendto() according to the chosen strategy. One **Receiver (Receiver.c)** accepts the frames that the channel forwards and acknowledges them. Reliable delivery per station uses the Stop-and-Wait ARQ of Assignment 2, and framing, checksum and CRC come from Assignments 1 and 2. Each station reports its average MAC delay, total time and throughput at the end.

2.2 Frame and ACK format

The frame is the one of Assignment 2 with CRC-8: a 16-byte header (source MAC, destination MAC, length, 16-bit sequence number), a 46-byte payload, a 2-byte checksum and a 1-byte CRC-8, 65 bytes in total. The Receiver accepts only datagrams of exactly 65 bytes. The ACK is 2 bytes (16-bit acknowledgement number, 0 or 1 in Stop-and-Wait). Control messages are short strings: a station sends SENSE and the Channel answers IDLE, BUSY or COLLISION.

2.3 Channel model

State	Meaning in Channel.c
IDLE	No frame in the last 50 ms window. A data frame arriving now moves the channel to BUSY, starts a 50 ms window and is forwarded to the Receiver
BUSY	A frame is occupying the medium for 50 ms (TRANS_DELAY_MS). A SENSE request is answered BUSY
COLLISION	A data frame arrived while the channel was BUSY or already in COLLISION. The collision counter increases, the 50 ms window restarts, and the late frame is damaged (byte 16 is inverted) before being forwarded. SENSE is answered COLLISION

When the window expires the state returns to IDLE. ACKs sent by the Receiver (port 4456) are relayed to the last station that transmitted.

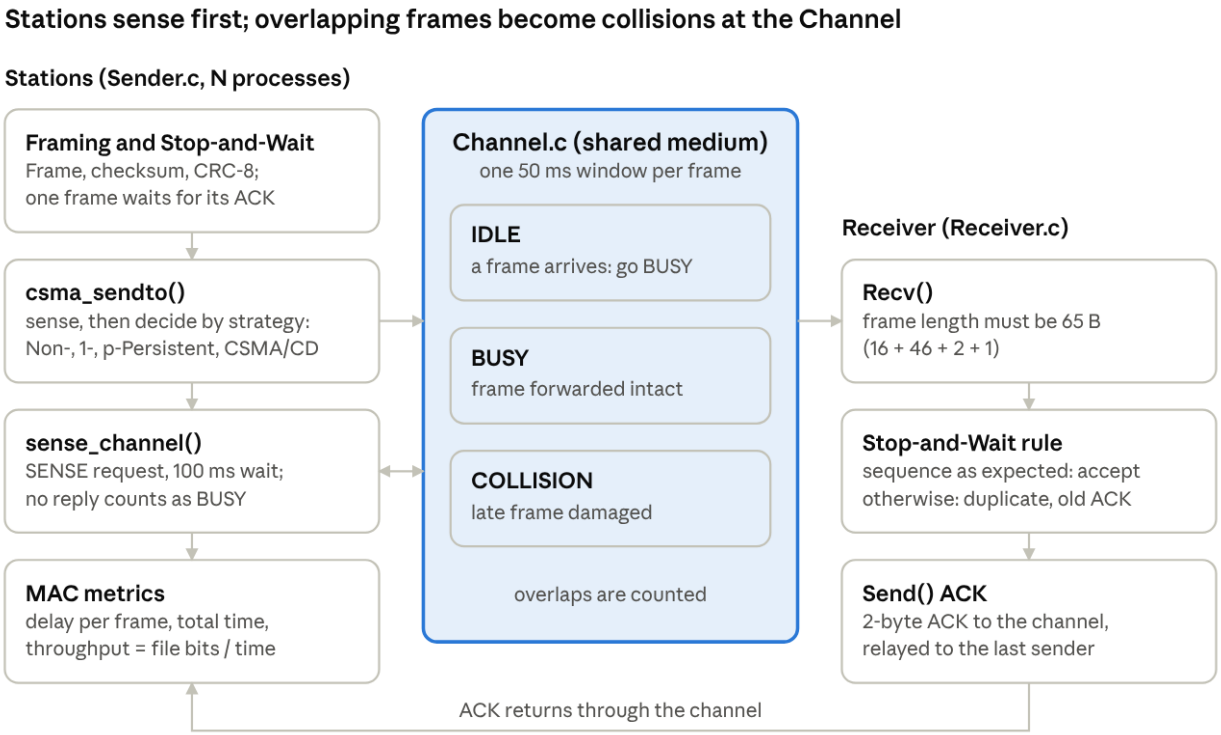
2.4 Station strategies

Strategy	Channel sensed IDLE	Channel sensed BUSY or COLLISION
Non-Persistent	Transmit	Wait a random 0 to 99 ms, then sense again
1-Persistent	Transmit	Sense again every 5 ms and transmit as soon as it is IDLE
p-Persistent	Transmit with probability p; otherwise wait one 50 ms slot and sense again	Sense again every 5 ms
CSMA/CD	As implemented, same as 1-Persistent (see Section 6)	Same as 1-Persistent

A station that gets no reply to `SENSE` within 100 ms treats the channel as busy.

2.5 Structure diagram

Stations sense and send, the Channel decides IDLE, BUSY or COLLISION for every frame, and the Receiver acknowledges what the Channel forwards.



structure diagram · stations, channel and receiver

Figure 1. Procedural organisation of the stations, the Channel and the Receiver.

2.6 Input and output format

- **Channel:** ./channel <Receiver_IP>. It runs until stopped and prints a line for each collision with a running total.
- **Receiver:** ./receiver, then the number of contending senders expected, the CRC width and the ARQ protocol. It ends when it has received a DONE datagram from every sender.
- **Sender (one process per station):** ./sender <Channel_IP> <Test_File>, then prompts for the CRC width, the error mode, the ARQ protocol (Stop-and-Wait), the CSMA strategy (1 = Non-Persistent, 2 = 1-Persistent, 3 = p-Persistent with its p, 4 = CSMA/CD).
- **Sender output:** a line per frame with its MAC delay in microseconds, and at the end the metrics block: total time (s), throughput (bps) and average MAC delay (ms).
- **Test data:** Test_f3fc9b.txt, 5,481 bytes, which gives 120 frames per station (119 full frames and one zero-padded frame).

3. Implementation

3.1 Source files

File	Role
Channel.c	Shared medium: IDLE, BUSY, COLLISION state, `SENSE` replies, collision counting, forwarding to the Receiver, ACK relay
Sender.c	Station: `sense_channel()`, `csma_sendto()`, strategy menu, metric computation
stop_wait_1.h	`send_stop_and_wait()` calling `csma_sendto()` for every transmission; 400 ms ACK wait
Receiver.c	Accepts 65-byte frames, applies the Stop-and-Wait sequence rule and sends the ACK
checksum_46c6c0.h, Crc_589259.h	Checksum and CRC modules from Assignment 1

3.2 Methods required by the assignment

Required behaviour	Implementation
Carrier sensing	`sense_channel()` sends `SENSE`, waits up to 100 ms and returns 0 (IDLE), 1 (BUSY) or 2 (COLLISION); no reply counts as busy
Persistence and back-off	`csma_sendto()`: Non-Persistent waits `rand() % 100` ms, 1-Persistent re-senses every 5 ms, p-Persistent draws a random number against p and defers 50 ms on failure
Collision detection	Done by the Channel: a data frame arriving during a window sets COLLISION, increments the collision counter and restarts the window
Metrics	MAC delay measured around each `csma_sendto()` call; total time and throughput (file bits divided by total time) measured around the whole transfer

3.3 Code snippets

Sensing and strategy decision (Sender.c):

```
while (1) {
    int status = sense_channel(sockfd, channel_addr);   /* 0 idle, 1 busy, 2
collision */
    if (status == 0) {
        if (csma_strategy == 3) {                       /* p-Persistent */
            if (((float)rand() / RAND_MAX) <= csma_p_prob) { sendto(...); break; }
            else { usleep(50000); continue; }           /* defer one slot */
        } else { sendto(...); break; }                  /* 1-Persistent, Non-
Persistent, CSMA/CD */
    } else {
        if (csma_strategy == 1) usleep((rand() % 100) * 1000);   /* Non-Persistent */
        else usleep(5000);                                       /* keep sensing */
    }
}
```

Collision handling in the Channel (Channel.c):

```
if (state == IDLE) {
    state = BUSY;  busy_until = now + TRANS_DELAY_MS;
    sendto(sockfd, buffer, bytes, 0, receiver_addr, ...);        /* forwarded intact
*/
} else {
    state = COLLISION;  total_collisions++;
    busy_until = now + TRANS_DELAY_MS;
    buffer[16] ^= 0xFF;                                          /* late frame
damaged */
    sendto(sockfd, buffer, bytes, 0, receiver_addr, ...);
}
```

4. Test Cases

All tests use `Test_f3fc9b.txt` (120 frames per station), Stop-and-Wait ARQ and error mode 1.

#	Test	Setting	What is checked
T1	p-Persistent against p	N = 3, p = 0.2, 0.5, 0.8	Collisions, average MAC delay and throughput as p grows
T2	All strategies against N	N = 2, 3, 4, 5; Non-Persistent, 1-Persistent, p-Persistent (p = 0.5), CSMA/CD	Collisions, delay and throughput of each scheme
T3	Single station	N = 1	No collisions and the minimum MAC delay (reference point)
T4	Busy channel	Second station starts while the first is transmitting	`SENSE` returns BUSY and the strategy-specific waiting behaviour appears
T5	Overlapping frames	Two stations sense IDLE in the same instant	Channel prints a collision and the counter increases
T6	Lost `SENSE` reply	Channel stopped or slow	Station waits 100 ms and treats the channel as busy
T7	ACK path	Stop-and-Wait with N stations	Frame is ACKed, sequence number toggles, `DONE` ends the Receiver after N senders

T1 and T2 produced the measurements of Section 5; T3 to T7 are checked from the console logs.

5. Results

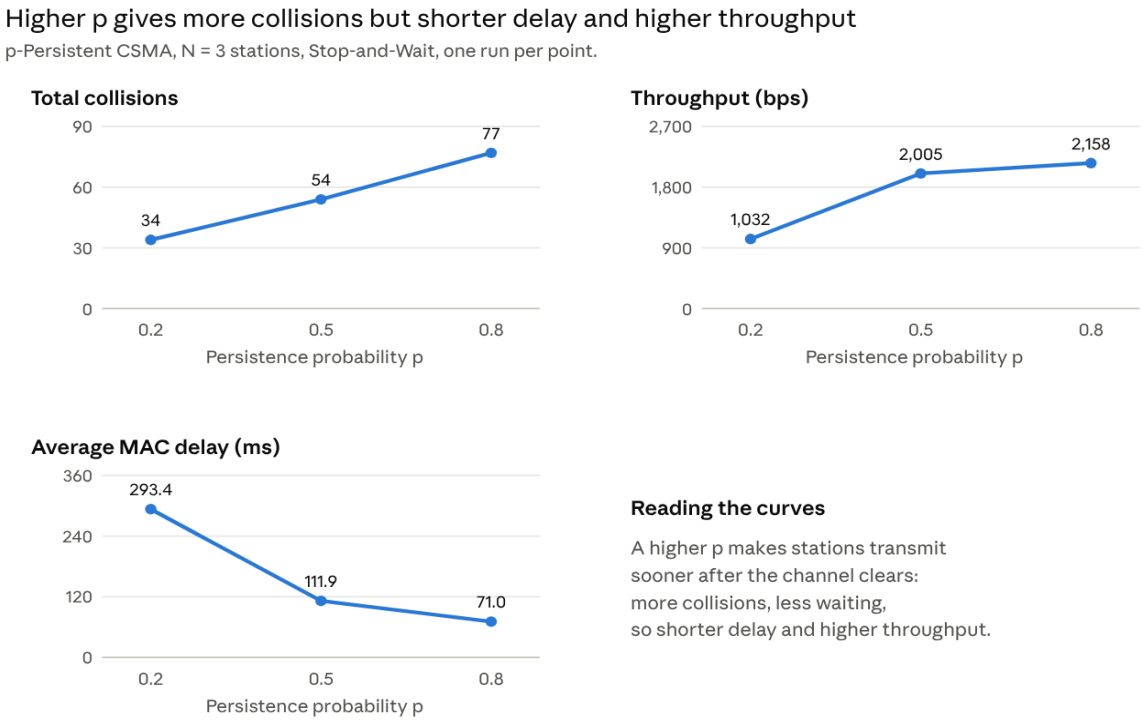
5.1 Setup and metrics

The measurements are those recorded in `Assignment_3.xlsx`, one run per point.

- **Total collisions:** the Channel's counter of data frames that arrived while the channel was already occupied.
- **Average MAC delay (ms):** mean time per frame from the first carrier sense to the moment the frame is handed to the channel.
- **Throughput (bps):** file size in bits (43,848) divided by the station's total transfer time, as printed by the Sender.

5.2 Experiment 1: p-Persistent CSMA against p (N = 3)

p	Total collisions	Average MAC delay (ms)	Throughput (bps)
0.2	34	293.35	1,032.26
0.5	54	111.90	2,004.54
0.8	77	70.98	2,158.45



graph set 1 · Assignment_3.xlsx, Experiment 1

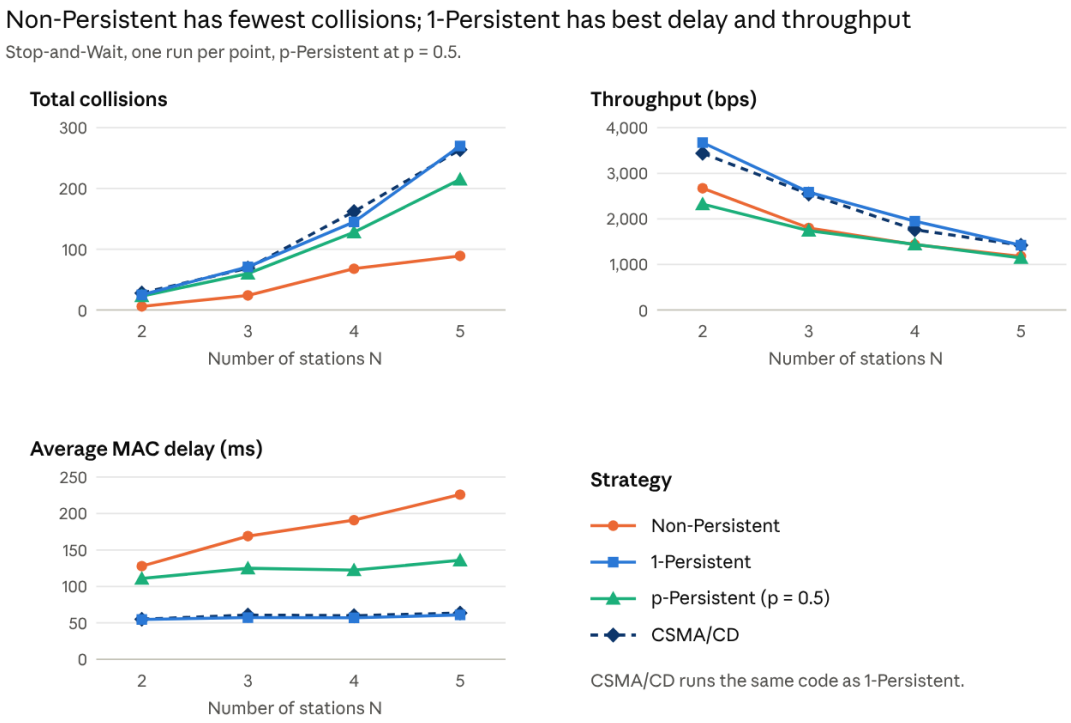
Figure 2. Collisions, throughput and average MAC delay against p for p-Persistent CSMA (N = 3).

Going from p = 0.2 to p = 0.8 raised collisions by 126% (34 to 77), cut the average MAC delay by 76% (293.35 to 70.98 ms) and raised throughput by 109% (1,032 to 2,158 bps). Most of the gain came before p = 0.5: from 0.5 to 0.8 the delay fell 37% and throughput rose 8%, while collisions rose 43%.

5.3 Experiment 2: all schemes against the number of stations N

Collisions per frame offered assumes that each station sends the whole 120-frame file, so N x 120 frames are offered.

N	Strategy	Total collisions	Collisions per frame offered	Average MAC delay (ms)	Throughput (bps)
2	Non-Persistent	6	2.5%	127.84	2,671.5
2	1-Persistent	25	10.4%	54.67	3,674.5
2	p-Persistent (p = 0.5)	23	9.6%	110.80	2,324.1
2	CSMA/CD	28	11.7%	55.00	3,440.1
3	Non-Persistent	24	6.7%	168.98	1,800.7
3	1-Persistent	71	19.7%	57.21	2,586.5
3	p-Persistent (p = 0.5)	60	16.7%	124.90	1,743.8
3	CSMA/CD	69	19.2%	60.78	2,546.3
4	Non-Persistent	68	14.2%	190.94	1,436.0
4	1-Persistent	145	30.2%	56.93	1,949.4
4	p-Persistent (p = 0.5)	128	26.7%	122.36	1,439.9
4	CSMA/CD	162	33.8%	59.90	1,763.4
5	Non-Persistent	89	14.8%	225.99	1,177.3
5	1-Persistent	270	45.0%	60.99	1,423.4
5	p-Persistent (p = 0.5)	215	35.8%	135.87	1,145.2
5	CSMA/CD	264	44.0%	63.50	1,421.9



graph set 2 · Assignment_3.xlsx, Experiment 2

Figure 3. Collisions, throughput and average MAC delay against the number of stations N for the four schemes.

Going from N = 2 to N = 5, collisions grew 9 to 15 times, per-station throughput fell by 51% to 61% depending on the scheme (Non-Persistent 56%, 1-Persistent 61%, p-Persistent 51%, CSMA/CD 59%), and the Non-Persistent delay rose 1.8 times while the 1-Persistent delay rose only 1.1 times.

6. Analysis

Effect of p (Experiment 1). Collisions rise with p because more waiting stations transmit the moment the channel clears. The delay falls and throughput rises with p because a station that defers loses a whole 50 ms slot every time it draws a failure; at p = 0.2 each frame waits on average several slots. Theory (the success probability N p (1 - p)^(N-1) is highest at p = 1/N, about 0.33 for N = 3) suggests a peak between 0.2 and 0.5 followed by a fall. The measured throughput keeps rising up to 0.8, because in this emulation a collision costs nothing extra (see the

limitations below). The sheet's label "best p = 0.5" is a compromise: p = 0.8 has the highest throughput and lowest delay, but 43% more collisions than p = 0.5.

Comparison of the schemes (Experiment 3).

1. **Non-Persistent** had the fewest collisions at every N (66% to 76% fewer than 1-Persistent at N = 3 to 5), but the longest delays (128 ms at N = 2, 226 ms at N = 5) and lower throughput, since a station that sees a busy channel sleeps up to 99 ms even if the channel frees sooner.
2. **1-Persistent** had the shortest delay (55 to 61 ms) and the highest throughput at every N, with the most collisions (45% of frames offered at N = 5).
3. **p-Persistent (p = 0.5)** had 8% to 20% fewer collisions than 1-Persistent, but about twice its delay and about 30% lower throughput, because every deferral costs a 50 ms slot.
4. **CSMA/CD** was within 12% of 1-Persistent in collisions and 10% in throughput at every N, in both directions. This is expected, because it runs the same code (limitation 1).

Why more collisions did not reduce throughput. In textbook CSMA a collision destroys the frames and forces retransmissions, so the scheme with the most collisions loses the most throughput. Here the Channel forwards the damaged frame and the Receiver does not check its checksum or CRC, so the colliding frame is still acknowledged and no retransmission happens. A collision only extends the busy window. That is why 1-Persistent, the scheme with the most collisions, delivers the best throughput and delay, and why waiting (Non-Persistent, p-Persistent) only adds idle time.

Variation between runs. Each point is one run. The N = 3, p = 0.5 configuration appears twice: 54 collisions, 111.9 ms and 2,004.5 bps in Experiment 1, and 60 collisions, 124.9 ms and 1,743.8 bps in Experiment 2, differences of 11%, 12% and 13%. Gaps smaller than this between schemes, such as 1-Persistent against CSMA/CD, should not be read as real.

Known bugs and limitations.

1. CSMA/CD is not implemented. Strategy 4 takes the same path as 1-Persistent: there is no listen-while-transmitting, no jam signal, no abort and no exponential back-off. The CSMA/CD rows are a second measurement of 1-Persistent.
2. Collisions are not harmful in the emulation: the Channel forwards the damaged frame, and Receiver.c checks only the length and the sequence number, never the checksum or CRC, and ACKs the frame. Frames are therefore never lost or retransmitted because of a collision.
3. The Receiver keeps one `expected_seq` for all stations, and the Channel relays every ACK to the last station that transmitted. With more than one station, ACKs can go to the wrong station and frames can be counted as duplicates, so some timeouts (400 ms) and retransmissions come from this mix-up and not from the CSMA strategy.
4. The MAC delay covers only the time to acquire the channel, not the ACK wait, so it does not include these timeouts, while the throughput does.
5. Only CRC-8 is implemented in `stop_wait_1.h`; choosing another CRC width sends a frame the Receiver ignores.
6. The 50 ms busy window and the 50 ms slot are fixed, there is no propagation delay model, and the only vulnerable period is the gap between the `SENSE` reply and the arrival of the frame.
7. Few points were measured: three values of p, N from 2 to 5, one run each. The throughput column appears to be a mean over stations, which should be stated in the demo.

Possible improvements. Implement real CSMA/CD in the sender (listen while sending, jam, abort, truncated binary exponential back-off, drop after 16 attempts); make the Channel drop colliding frames and make the Receiver verify checksum and CRC so that collisions force retransmissions; keep per-station sequence numbers and route ACKs by station; measure more values of p (for example 0.05 to 1.0 in steps of 0.05, which would show the peak near 1/N), larger N and several runs per point.

7. Comments

This assignment showed how the persistence rule of a station trades idle time against collisions, and how much the result depends on what a collision costs in the model. Running several independent sender processes against one channel emulator was the hardest part, because ACK routing and sequence numbers had to work for several stations at once. The assignment was of moderate difficulty. A useful improvement would be to state exactly what happens to colliding frames, so that the emulated channel and the textbook model can be compared directly.

Appendix: source files

Channel.c, Sender.c, Receiver.c, stop_wait_1.h, checksum_46c6c0.h, Crc_589259.h, Test_f3fc9b.txt (5,481 bytes), Assignment_3.xlsx (measurements).